

*Operating
Systems:
Internals
and Design
Principles*

Chapter 7 Memory Management

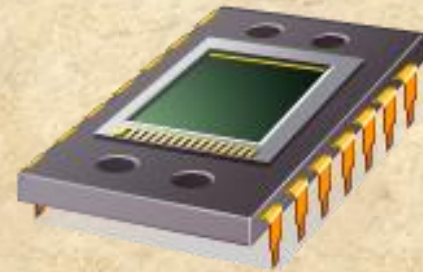
Eighth Edition
William Stallings

Memory Management

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly.
- Subdividing memory to accommodate multiple processes
- Memory needs to be allocated to ensure a reasonable supply of ready processes to consume available processor time

Memory Management Requirements

- Memory management is intended to satisfy the following requirements:
 - Relocation
 - Protection
 - Sharing
 - Logical organization
 - Physical organization



Relocation

- Programmer does not know where the program will be placed in memory when it is executed.
- Active processes need to be able to be swapped in and out of main memory in order to maximize processor utilization.
- Specifying that a process must be placed in the same memory region when it is swapped back in would be limiting the performance.
 - While the program is executing, it may be swapped to disk and returned to main memory at a different location (relocated)

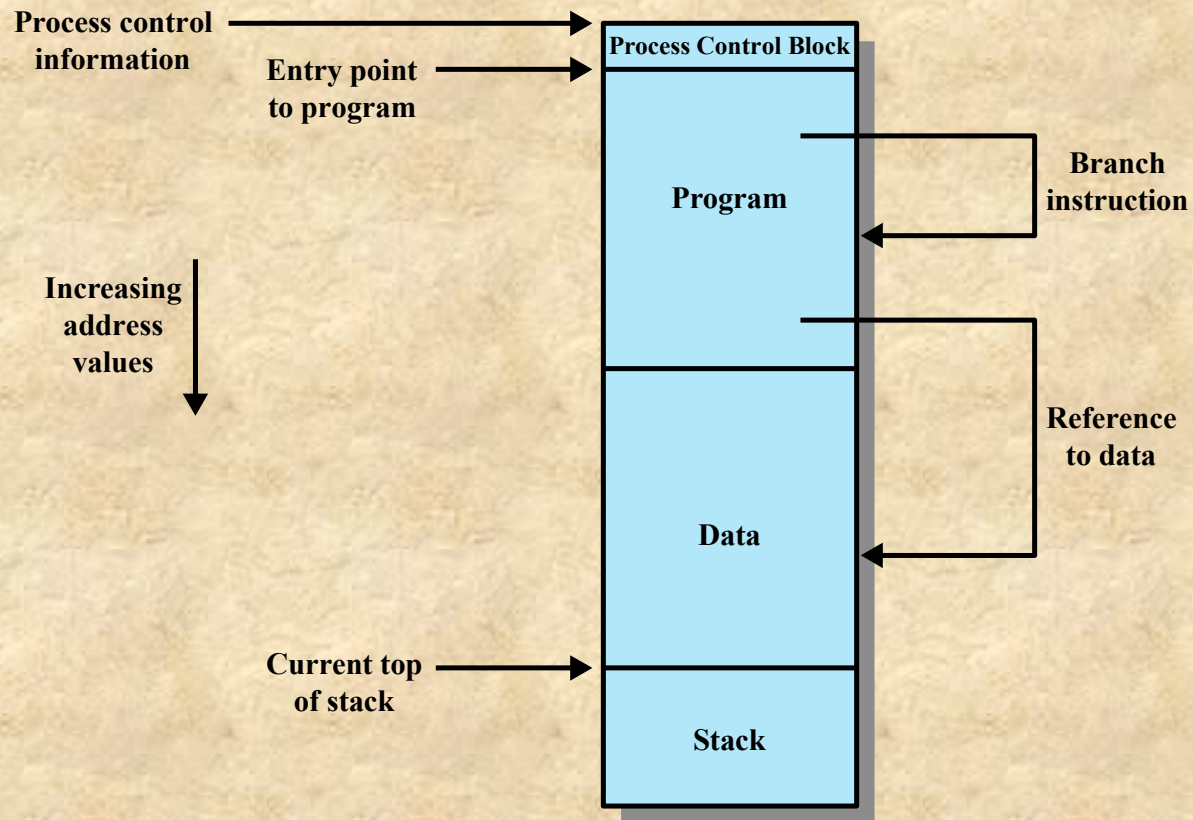


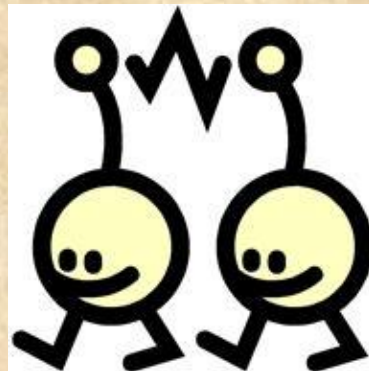
Figure 7.1 Addressing Requirements for a Process

Protection

- Processes need to acquire permission to reference memory locations for reading or writing purposes.
- Location of a program in main memory is unpredictable.
- Memory references generated by a process must be checked at run time.
 - Impossible to check absolute addresses at compile time.
- Mechanisms that support relocation also support protection.
- Memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software).
 - Operating system cannot anticipate all of the memory references a program will make.

Sharing

- Allow several processes to access the same portion of memory
- Better to allow each process access to the same copy of the program rather than have their own separate copy



Logical Organization

- Memory is organized as linear

Programs are written in modules

- modules can be written and compiled independently
- different degrees of protection given to modules (read-only, execute-only)
- sharing on a module level corresponds to the user's way of viewing the problem

Physical Organization

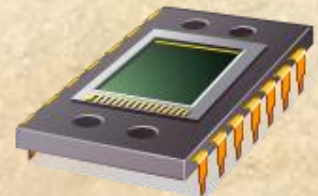
- The flow of information between main and secondary memory is major system concern.
- Cannot leave the programmer with the responsibility to manage memory.
 - Reasons:

The main memory available for a program plus its data may be insufficient.

In multiprogramming environment, programmer does not know how much space will be available.

Memory Management

- Memory management brings processes into main memory for execution by the processor.
- Two techniques:
 - Involves virtual memory
 - Segmentation
 - Paging
 - Do not involve virtual memory
 - Partitioning



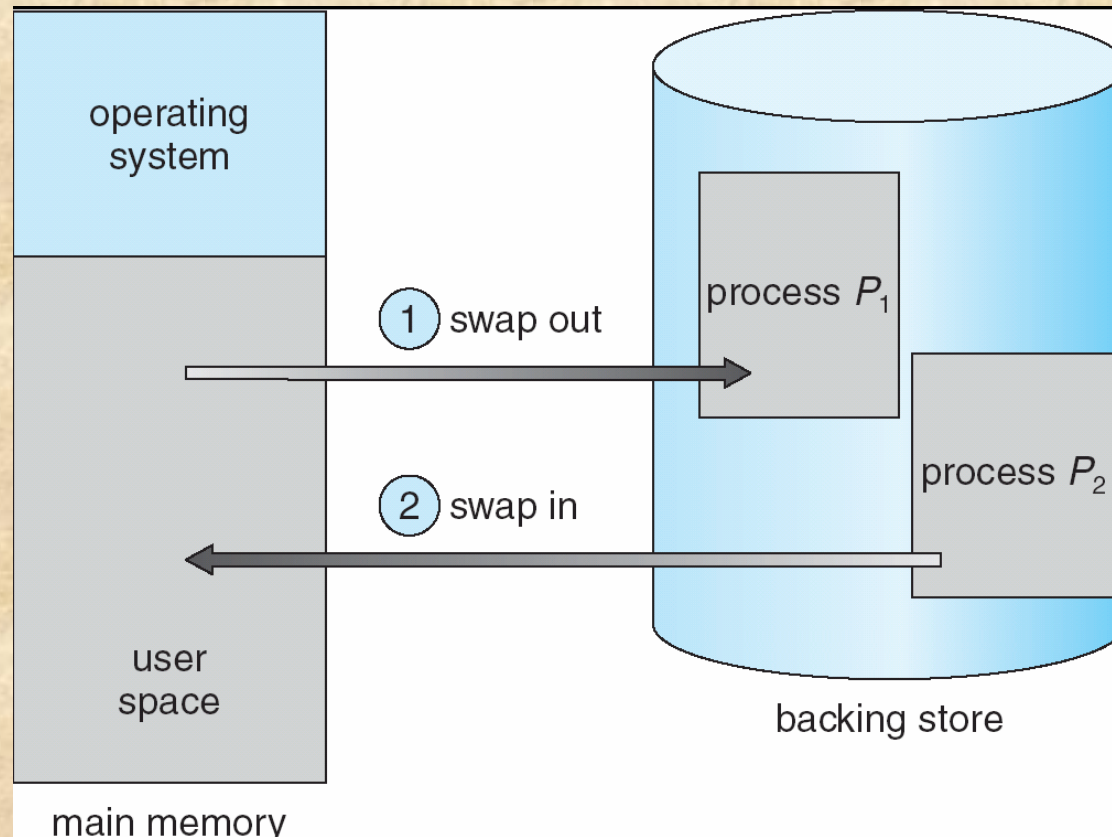
Swapping

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution.
- **Backing store**—fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images.
- **Roll out, roll in**—swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed.

Swapping

- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped.
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows).
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk.

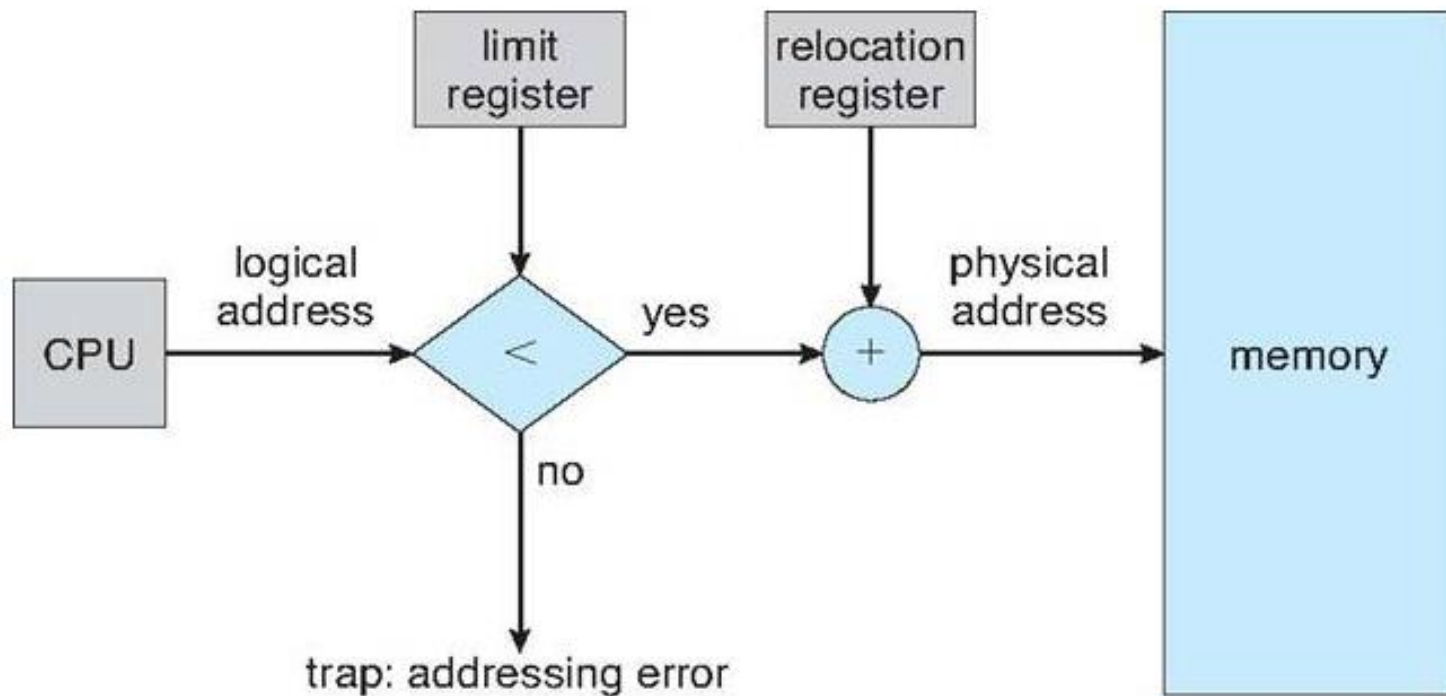
Schematic View of Swapping



Contiguous Memory Allocation

- Main memory usually into two partitions:
 - Resident operating system, usually held in low memory with interrupt vector.
 - User processes then held in high memory.
- Relocation registers used to protect user processes from each other, and from changing operating-system code and data.
- Base register contains value of smallest physical address.
- Limit register contains range of logical addresses –each logical address must be less than the limit register .
- MMU maps logical address *dynamically*.

HW address protection with base & limit register



Contiguous Memory Allocation

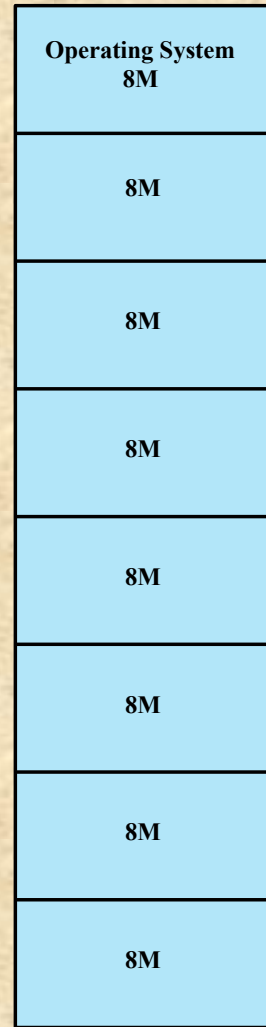
- Multiple-partition allocation
 - Hole –block of available memory; holes of various size are scattered throughout memory.
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it.
 - Operating system maintains information about:
 - a) allocated partitions b) free partitions (hole)

Fixed Partitioning

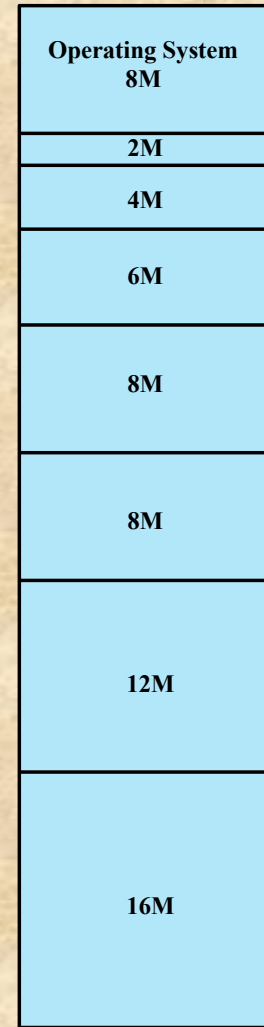
- Any process whose size is less than or equal to the partition size can be loaded into an available partition
- If all partitions are full, the operating system can swap a process out of a partition
- A program may not fit in a partition. The programmer must design the program with overlays
- Two types:
 - Equal size
 - Unequal size

Fixed Partitioning

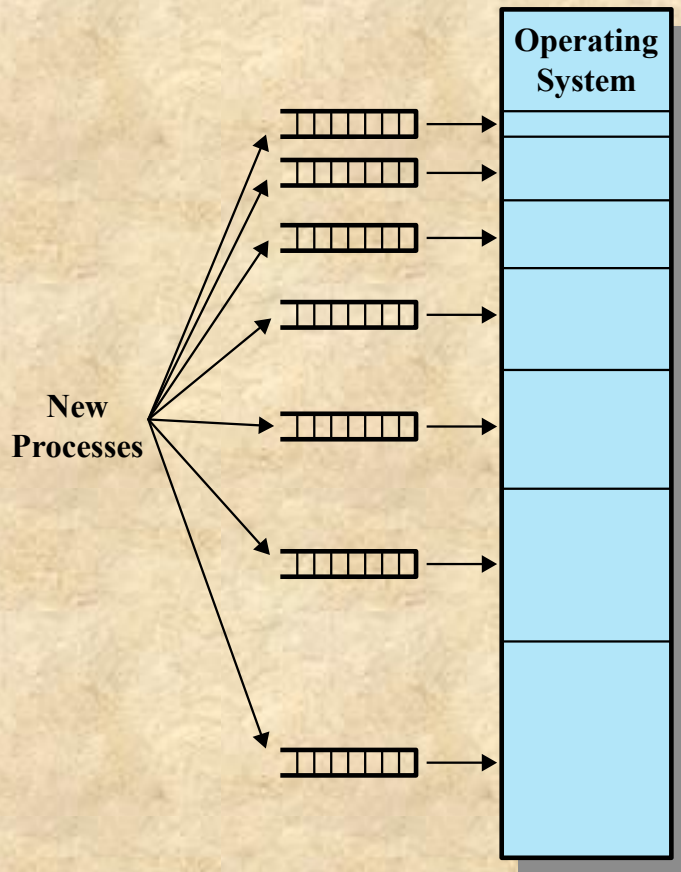
- Equal-size partitions
 - Because all partitions are of equal size, it does not matter which partition is used
- Unequal-size partitions
 - Can assign each process to the smallest partition within which it will fit
 - Queue for each partition
 - Processes are assigned in such a way as to minimize wasted memory within a partition



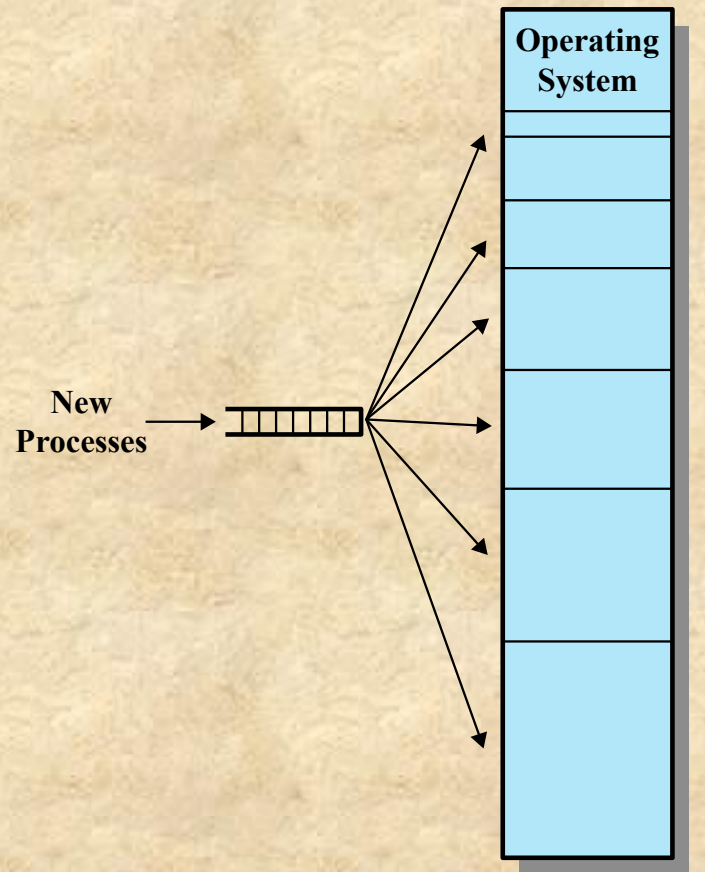
(a) Equal-size partitions



(b) Unequal-size partitions



(a) One process queue per partition



(b) Single queue

Figure 7.3 Memory Assignment for Fixed Partitioning

Disadvantages

- The number of partitions specified at system generation time limits the number of active processes in the system
- Small jobs will not utilize partition space efficiently



Disadvantages

- A program may be too big to fit in a partition
 - program needs to be designed with the use of overlays
- Main memory utilization is inefficient
 - any program, regardless of size, occupies an entire partition. This is called **internal fragmentation**.
 - *internal fragmentation*
 - wasted space due to the block of data loaded being smaller than the partition



Dynamic Partitioning

- Partitions are of variable length and number.
- Process is allocated exactly as much memory as it requires.
- Eventually get holes in the memory. This is called **external fragmentation**.
- Must use compaction to shift processes so they are contiguous and all free memory is in one block.

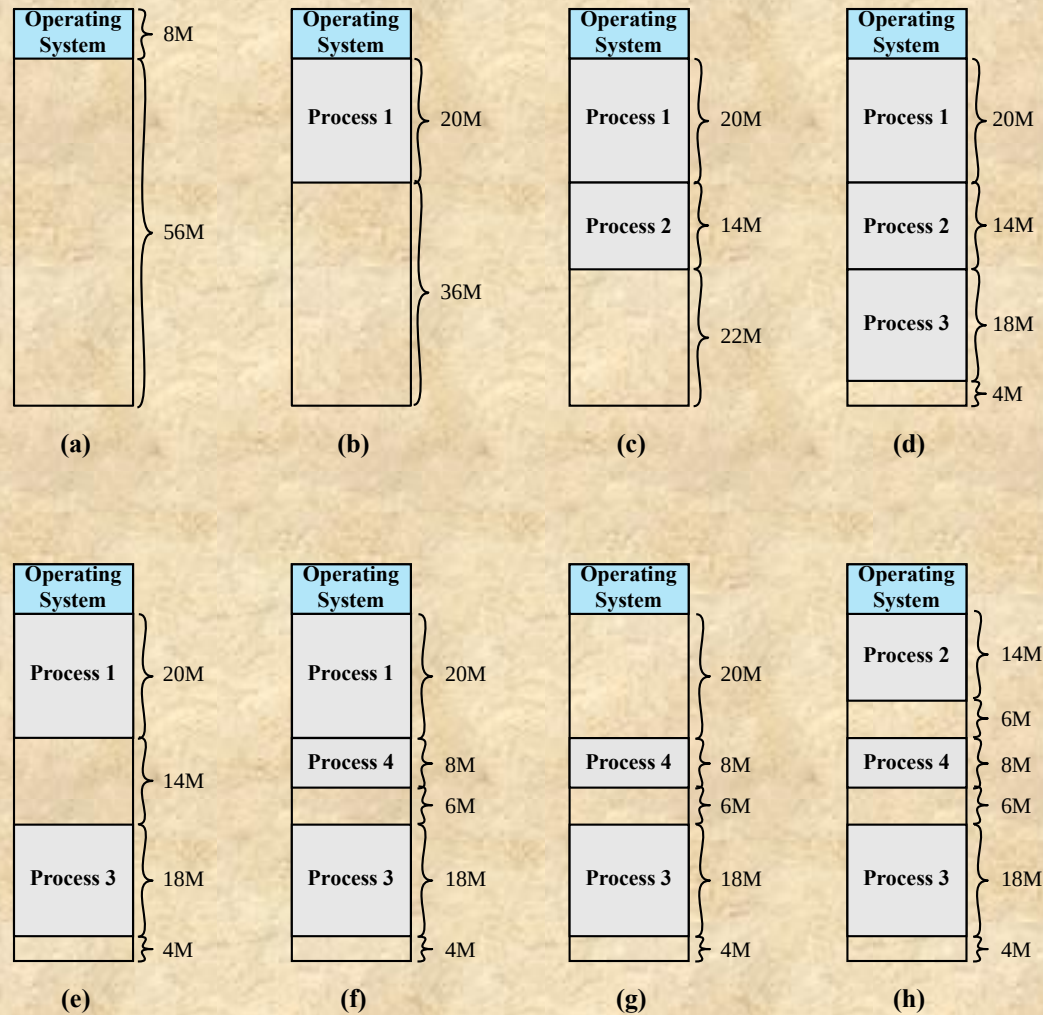


Figure 7.4 The Effect of Dynamic Partitioning

Dynamic Partitioning

External Fragmentation

- memory becomes more and more fragmented
- memory utilization declines

Compaction

- technique for overcoming external fragmentation
- OS shifts processes so that they are contiguous
- free memory is together in one block
- time consuming and wastes CPU time

Dynamic Partitioning Placement Algorithm

- Operating system must decide which free block to allocate to a process.
- **Best-fit algorithm**
 - Chooses the block that is closest in size to the request.
 - Worst performer overall.
 - Since smallest block is found for process, the smallest amount of fragmentation is left.
 - Memory compaction must be done more often.

Dynamic Partitioning Placement Algorithm

■ First-fit algorithm

- Scans memory from the beginning and chooses the first available block that is large enough.
- Fastest.
- May have many process loaded in the front end of memory that must be searched over when trying to find a free block.

Dynamic Partitioning Placement Algorithm

■ Worst-fit

- Scans memory from the location of the last placement
- More often allocate a block of memory at the end of memory where the largest block is found
- The largest block of memory is broken up into smaller blocks
- Compaction is required to obtain a large block at the end of memory

Exercise

Given a heap memory management scheme with the following free list:

- U - Used
- H - Hole

U	H	H	U	U	H	U	H	H	U	U	H	
0	10	19	40	60	100	110	123	178	200	248	279	460

The following process request will be received in order:

Process Number	Size in Kilobytes
1	13
2	51
3	27
4	83
5	9

Exercise (Cont'd)

Show how the memory request above is allocated using each of the Dynamic Partitioning Placement Algorithms:

- **First Fit (FF)**
- **Worst Fit (WF)**
- **Best Fit (BF)**

Logical vs. Physical Address Space

■ Logical

- Generated by CPU.
- Reference to a memory location independent of the current assignment of data to memory.
- Translation must be made to the physical address.

■ Physical

- The absolute address or actual location in main memory.

Registers Used during Execution

- Base register
 - Starting address for the process.
- Bounds register
 - Ending location of the process.
- These values are set when the process is loaded or when the process is swapped in.

Paging

- Partition memory into small equal fixed-size chunks and divide each process into the same size chunks
- The chunks of a process are called pages and chunks of memory are called frames
- Operating system maintains a page table for each process
 - Contains the frame location for each page in the process
 - Memory address consist of a page number and offset within the page

Assignment of Process Pages to Free Frames

Frame number	Main memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

(a) Fifteen Available Frames

	Main memory
0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

(b) Load Process A

	Main memory
0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	
8	
9	
10	
11	
12	
13	
14	

(c) Load Process B

Assignment of Process Pages to Free Frames

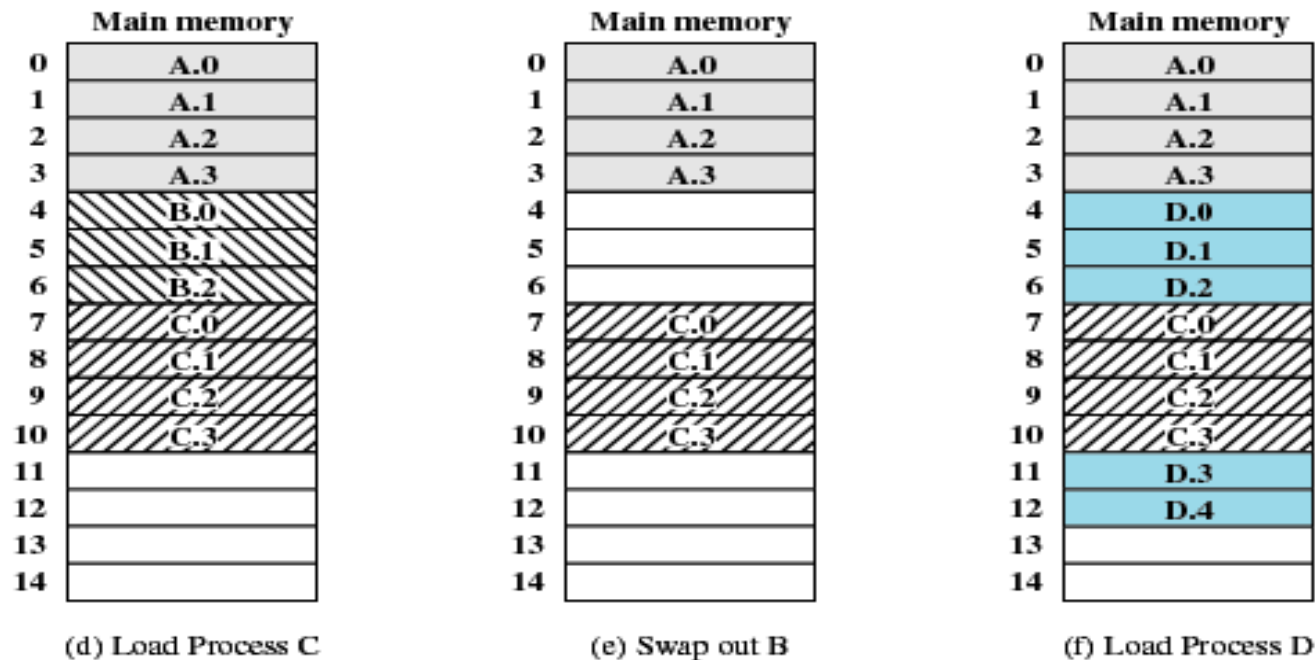


Figure 7.9 Assignment of Process Pages to Free Frames

Segmentation

- A program can be subdivided into segments
 - may vary in length
 - there is a maximum length
- Addressing consists of two parts:
 - segment number
 - an offset
- Since segments are not equal, segmentation is similar to dynamic partitioning

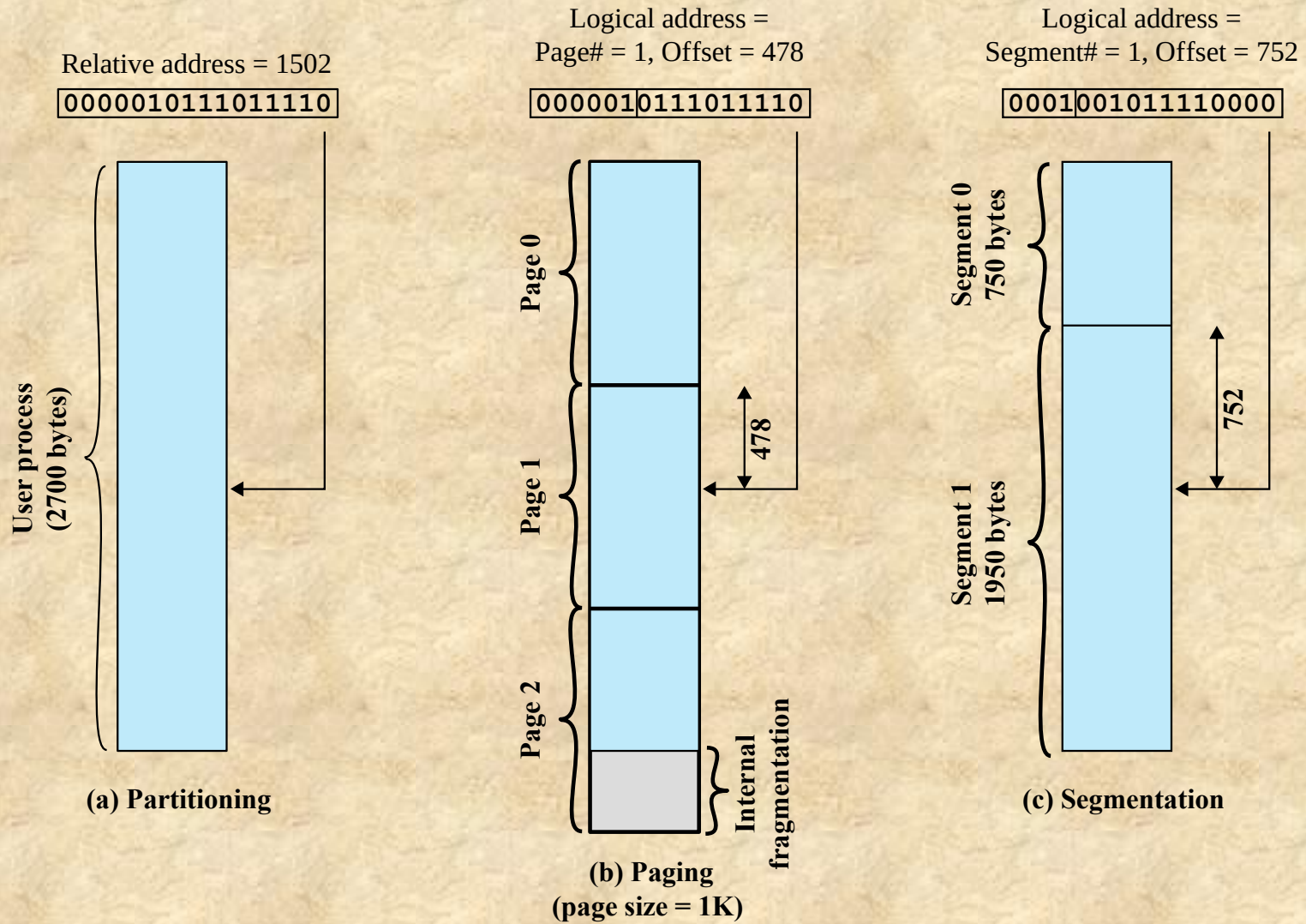


Figure 7.11 Logical Addresses

Summary

- Memory management requirements
 - relocation
 - protection
 - sharing
 - logical organization
 - physical organization
- Paging
- Memory partitioning
 - Swapping
 - Contiguous memory allocation
 - fixed partitioning
 - dynamic partitioning
- Segmentation